

핵심 정리

8장 CNN

- 컨볼루션 신경망 (CNN)이란?
 - 완전 연결 신경망(Fully-connected network)의 한계점
 - CNN의 등장 및 주요 특징
- CNN의 핵심 구성 요소
 - 컨볼루션 (Convolution)
 - 풀링 (Pooling)
- 손실 함수 (Loss Function)
- Cifar-10 예제 및 Keras 구현
 - Cifar-10 분류를 위한 CNN 모델
- 영상 인식 시스템 고급 기술 및 CNN 모델
 - 정규화 기법 (Regularization Techniques)
 - 대표적인 CNN 모델
- 전이 학습 (Transfer Learning)
 - VGG16을 이용한 개-고양이 인식 예시 (전이 학습)

9장 A 인식 - Detection

- 인식 관련 문제: 인공지능이 '보는' 방법
 - 분류 (Classification)
 - 검출 (Detection)
 - 분할 (Segmentation)
- 인식 데이터셋: 인공지능 학습을 위한 자료
- 검출 (Object Detection): 물체 위치와 종류 찾아내기
 - 성능 척도 (Performance Metrics)
 - 사물 영역 탐지에서의 문제점
 - 다중 사물 영역 탐지 기법
- 주요 사물 탐지 기술: 딥러닝의 발전
 - R-CNN (Region-based CNN, 2014)
 - Fast R-CNN
 - Faster R-CNN
 - YOLO (You Only Look Once)

9장 B 인식 - Segmentation

- 영상 분할이란 무엇인가요?
- 영상 분할의 주요 종류
- 영상 분할 모델의 성능은 어떻게 측정하나요?
- FCN (Fully Convolutional Networks): 영상 분할의 시작점
 - FCN 등장 이전의 문제점
 - FCN의 혁신
 - FCN의 구조와 동작 원리
 - 전치 컨볼루션(Transposed Convolution) 상세 설명
- FCN을 개선한 모델들

10장 동적 비전

- 동적 비전이란 무엇인가?
 - 동적 비전의 정의
 - 주요 활용 분야
 - 응용 분야의 제한점
 - 비디오(동영상)의 이해
- 움직이는 부분을 찾는 방법: 차이 영상(Difference Image)
 - 차이 영상의 개념
 - 차이 영상 계산 원리
 - 차이 영상 계산 절차 상세
 - 차이 영상의 한계
- 물체의 움직임 추적: 광류(Optical Flow)

- 광류의 개념
- 광류의 종류: 특징점 vs. 전체 영역
 - 특징점에 대한 광류
 - Dense Optical Flow (전체 영역 광류)
밝기 항상성(Brightness Constancy) 가정
광류 계산식 유도
- 광류 계산의 문제점과 해결 방법
 - 문제점
 - 해결을 위한 추가 가정
 - Smoothness 조건
- 최신 기술 동향: 딥러닝 기반 광류 추정
 - 참값 레이블링의 어려움
 - 주요 데이터셋
 - 딥러닝 적용 사례
- 광류의 실제 활용 사례

12장 3차원 비전

- 3차원 공간의 중요성과 특성
- 카메라 캘리브레이션: 3차원 정보 추출의 핵심
 - 카메라 기하 구조: 썬 렌즈 광학 (Thin Lens Optics)
 - 조리개 제어 (Aperture Control)
 - 핀홀 카메라 모델 (Pinhole Camera Model) & 원근 투영 (Perspective Projection)
 - 3차원 기하 변환
 - 영상 생성 기하: 외부 행렬과 내부 행렬
 - 카메라 캘리브레이션의 의미 및 절차
- 깊이 추정: 물체까지의 거리 측정
 - 깊이 영상의 필요성
 - 깊이 영상 획득 방법
 - 스테레오 비전
 - 밀집(Dense) 깊이 영상
 - 능동 센서
 - 점 구름 (Point Cloud) 데이터

13장 생성 비전

- 비전 인공지능의 두 가지 축: 분별과 생성
 - 분별(Discriminative) 비전이란?
 - 생성(Generative) 비전이란?
- 생성 모델이란 무엇일까요?
 - 생성 모델의 기본 개념
 - 다름 가능(Tractable) vs. 다름 불가능(Intractable)
- 생성 모델의 발전 과정
- 다양한 생성 모델 살펴보기
 - 가우시안 혼합 모델 (Gaussian Mixture Model; GMM)
 - 오토인코더(Autoencoder)를 이용한 생성 모델
- 생성 적대 신경망 (GAN): 이미지 생성의 혁신
 - GAN의 목표
 - GAN의 동작 방식 (생성자와 판별자의 경쟁)
 - DC-GAN (Deep Convolutional GAN)
 - GAN 방식의 장점과 단점

8장 CNN

1. 컨볼루션 신경망 (CNN)이란?

컨볼루션 신경망(Convolutional Neural Networks, CNN)은 특히 **영상(이미지) 처리 분야**에서 뛰어난 성능을 보이는 인공지능망의 한 종류입니다. 기존의 일반적인 신경망인 '완전 연결 신경망(Fully-connected network)'이 가진 한계를 극복하기 위해 등

장했습니다.

1.1. 완전 연결 신경망(Fully-connected network)의 한계점

완전 연결 신경망은 다음과 같은 특징과 한계를 가집니다:

- 모든 뉴런 연결: 현 단계의 뉴런 출력을 계산하기 위해 이전 단계의 모든 뉴런을 입력으로 사용합니다.
- 계수(가중치) 폭증: 뉴런의 계수가 증가할수록 연결해야 할 계수(weights), 즉 학습해야 할 파라미터의 계수가 기하급수적으로 증가합니다. 이는 계산량을 늘리고 과적합(Overfitting)의 위험을 높입니다.

1.2. CNN의 등장 및 주요 특징

CNN은 이러한 완전 연결 신경망의 한계를 극복하고 영상 처리의 특성을 효과적으로 반영하기 위해 '지역적인 계산(local connection)' 방식을 도입했습니다.

CNN의 핵심적인 특징과 장점은 다음과 같습니다:

- 희소 상호작용 (Sparse interaction): 이전 계층의 모든 뉴런과 연결되는 것이 아니라, 특정 지역에 위치한 제한된 수의 셀(뉴런)하고만 연결됩니다. 예를 들어, 영상의 5x5 픽셀 영역에만 반응하는 식입니다. 이는 모델의 복잡성을 줄여줍니다.
- 매개변수 공유 (Parameter sharing): 영상의 모든 위치에 **동일한 컨볼루션 마스크(필터)**를 적용하여 특징을 추출합니다. 반면, 완전 연결망은 각 셀에 서로 다른 가중치를 사용합니다. 이 '공유' 덕분에 학습해야 할 파라미터(가중치)의 수가 크게 줄어듭니다.
- 파라미터 수 감소: 위 두 가지 특징 덕분에 CNN은 학습해야 할 파라미터(가중치)의 숫자가 매우 적어집니다.
- 영상 인식에 최적화: 이러한 구조적 특성 때문에 CNN은 영상 인식 분야에서 특히 많이 사용되고 있습니다.

2. CNN의 핵심 구성 요소

CNN은 주로 **컨볼루션 계층(Convolution layer)**과 **풀링 계층(Pooling layer)**으로 구성됩니다.

2.1. 컨볼루션 (Convolution)

컨볼루션은 CNN의 가장 기본적인 연산으로, 입력 영상에 **필터(Filter, 또는 마스크, 커널)**를 적용하여 영상의 특징을 추출하고 ***특징 맵(Feature Map)***을 생성하는 과정입니다.

- 연산 과정: 일정한 크기(예: 3x3 또는 5x5)의 필터가 입력 영상을 훑어가며(슬라이딩하며) 픽셀 값과 필터 값을 곱하고 모두 더하는 연산을 수행합니다. 이 연산 결과는 **비선형 활성화 함수**를 통해 출력됩니다.
- 활성화 함수 (Activation Function): 컨볼루션 연산 후에는 비선형성을 부여하기 위해 활성화 함수를 적용하는데, CNN에서는 주로 **ReLU(Rectified Linear Unit)** 함수가 사용됩니다.
- 다중 마스크/필터: 일반적으로 한 장의 필터만 사용하는 것이 아니라, **여러 장의 마스크(필터)**를 이용하여 여러 개의 특징 맵(출력 영상)을 생성합니다. 이는 마치 예지(경계선) 검출 시 수평 방향과 수직 방향의 마스크를 각각 사용하는 것과 유사합니다. 여러 필터를 사용하면 영상에서 **다양한 특성(예: 예지, 질감 등)**을 분석할 수 있습니다.
- 입력/필터의 깊이(Depth): 입력 영상(예: 컬러 영상은 R, G, B 3개 채널이므로 깊이가 3)과 필터는 **동일한 깊이**를 가져야 컨볼루션 연산을 수행할 수 있습니다. 컨볼루션 결과는 깊이가 1인 단일 특징 맵이 됩니다.
- 매개변수 공유 (Parameter sharing): 컨볼루션 연산을 수행할 때, 영상의 모든 위치에 **동일한 필터 계수**가 적용됩니다.
- 스트라이드 (Stride): 컨볼루션 연산을 모든 픽셀마다 수행하는 대신, **일정 간격으로 건너뛰면서** 수행하는 것을 '스트라이드'라고 합니다. 스트라이드를 사용하면 출력 영상의 크기가 줄어드는 **다운샘플링(Down-sampling)** 효과를 얻을 수 있습니다. CNN 구조에서 스트라이드를 사용하는 경우가 많습니다.

2.2. 풀링 (Pooling)

풀링은 컨볼루션 계층 다음에 일반적으로 오는 단계로, 특징 맵의 **해상도를 낮추고(다운샘플링)** 변수의 개수를 줄이는 역할을 합니다.

- 연산 과정: 이전 계층의 특정 크기 영역(예: 2x2 픽셀)을 **하나의 픽셀로 줄이는** 과정입니다.
- 주요 방식:
 - Max Pooling (최대 풀링):** 2x2 영역 내의 네 값 중 **최대값**을 취하여 하나의 픽셀로 만듭니다.
 - Average Pooling (평균 풀링):** 2x2 영역 내의 네 값의 **평균값**을 취하여 하나의 픽셀로 만듭니다.
- 풀링을 하는 이유:
 - 변수 개수 축소 (Downsampling):** 해상도를 낮춰 모델이 처리해야 할 데이터의 양과 파라미터 수를 줄여줍니다.
 - 이동 불변성 (Translation invariance) 도입:** 영상 내의 객체 위치가 약간 바뀌어도 동일하게 인식될 수 있도록 하는 효과를 가져옵니다. 예를 들어, 고양이의 눈이 1-2 픽셀 움직여도 여전히 고양이의 눈으로 인식되게 돕습니다.
- 풀링을 수행하면 영상의 크기가 줄어듭니다.

3. 손실 함수 (Loss Function)

손실 함수는 신경망 학습 과정에서 **모델의 예측 값과 실제 값 사이의 오차(차이)**를 측정하는 데 사용되는 함수입니다. 이 오차를 줄이는 방향으로 신경망이 학습됩니다.

- 초기 신경망: 초기 신경망에서는 주로 **평균제곱오차(Mean Squared Error, MSE)**를 손실 함수로 사용했습니다.
- 딥러닝: 딥러닝에서는 작업(Task)의 종류에 따라 다양한 손실 함수가 사용됩니다.
 - 분류(Classification) 문제: 주로 **교차 엔트로피(Cross-entropy)**를 사용합니다.
 - 다른 작업: 물체 검출, 분할, 추적 등과 같은 특수 작업에는 해당 작업에 최적화된 다양한 손실 함수가 사용될 수 있습니다.
- 교차 엔트로피란? 참값(정답) 벡터와 예측(모델의 출력) 벡터를 확률 분포로 간주하고, 이 두 **확률 분포가 얼마나 다른지**를 측정하는 함수입니다. 두 분포가 비슷할수록 교차 엔트로피 값은 낮아지고, 차이가 클수록 높아집니다.

4. Cifar-10 예제 및 Keras 구현

Cifar-10은 기본적인 영상 인식 프로그램의 성능을 평가하기 위해 캐나다 대학에서 2010년에 공개한 표준 데이터셋입니다.

- 데이터셋 특징:
 - 크기: 32x32 픽셀 크기의 컬러 영상으로 구성됩니다. 해상도가 그리 높지 않습니다.
 - 클래스: 총 10개의 클래스(예: 비행기, 자동차, 새, 고양이 등)로 나뉘어 있습니다.
 - 영상 수: 총 60,000장의 영상이 있으며, 이 중 50,000장은 훈련(train)용, 10,000장은 테스트(test)용으로 구성되어 있습니다.
 - 성능: 기본적인 CNN 구조로 구현했을 때 오차율은 약 25% 정도로 나타납니다. 가장 성능이 우수한 알고리즘은 1% 가량의 오차율을 보입니다.

4.1. Cifar-10 분류를 위한 CNN 모델 (Keras)

제공된 자료에는 Keras(텐서플로우 기반의 고수준 API)를 사용하여 Cifar-10을 분류하는 CNN 모델의 전체 프로그램 코드가 제시되어 있습니다.

- 신경망 구조:
 - Conv2D** (컨볼루션 계층)
 - MaxPooling2D** (최대 풀링 계층)
 - Flatten** (다차원 데이터를 1차원으로 펼치는 계층)
 - Dense** (완전 연결 계층)

- 이러한 계층들이 조합되어 구성됩니다.
- 예시 모델은 총 **122,570개**의 파라미터를 가집니다.
- **데이터 전처리:**
 - Cifar-10 데이터는 Keras의 `keras.datasets.cifar10.load_data()` 함수를 통해 쉽게 불러올 수 있습니다.
 - 원본 영상의 픽셀 값은 0에서 255 사이인데, 이를 모델 학습에 적합하도록 0에서 1 사이의 값으로 **정규화 (Normalization)**합니다.
- **모델 훈련:**
 - **컴파일(Compile):** `optimizer='adam'` (최적화 알고리즘), `loss='sparse_categorical_crossentropy'` (손실 함수), `metrics=['accuracy']` (성능 측정 지표)를 사용하여 모델을 컴파일합니다.
 - **훈련(Fit):** `model.fit()` 함수를 사용하여 훈련 데이터를 기반으로 모델을 학습시킵니다.
 - **에폭(Epochs):** 전체 훈련 데이터를 몇 번 반복하여 학습할지를 나타내는 '에폭'은 수렴을 위해 20번이 적절하지만, 예시에서는 시간 절약을 위해 4번으로 설정되었습니다.
 - **EarlyStopping:** `EarlyStopping` 콜백 함수를 사용하여 모델의 성능이 더 이상 개선되지 않을 때 훈련을 조기에 중단시켜 과적합을 방지합니다.

5. 영상 인식 시스템 고급 기술 및 CNN 모델

모델의 성능을 향상시키고 과적합(Overfitting)을 방지하기 위한 다양한 기법들과, 영상 인식 분야에서 중요한 역할을 한 대표적인 CNN 모델들이 소개됩니다.

5.1. 정규화 기법 (Regularization Techniques)

과적합은 모델이 훈련 데이터에 너무 맞춰져 새로운 데이터에 대한 성능이 떨어지는 현상을 말합니다. 이를 방지하기 위한 기법들입니다.

- **드롭아웃 (Dropout):**
 - **개념:** 학습 과정에서 신경망의 일부 뉴런을 **임의로 비활성화(사용하지 않게)**하여 훈련 시 과적합을 방지하는 기법입니다. 마치 여러 개의 작은 신경망을 앙상블하는 효과를 냅니다.
 - **Keras 구현:** `layers.Dropout()` 을 사용하여 원하는 제거 비율(예: 0.2, 0.3, 0.4)을 지정하여 레이어를 추가합니다.
- **배치 정규화 (Batch Normalization):**
 - **개념:** 훈련 과정을 안정화시키는 방법 중 하나로, 각 계층의 입력 데이터를 **평균 0, 표준편차 1**이 되도록 정규화하는 기법입니다. 이는 학습 속도를 향상시키고 더 깊은 신경망의 학습을 용이하게 합니다.
 - **Keras 구현:** `Conv2D` 계층 다음에 `layers.BatchNormalization()` 계층을 추가합니다.
- **데이터 증강 (Data Augmentation):**
 - **개념:** 학습할 샘플(데이터)의 수가 너무 적을 때, 원본 영상을 **약간 변형시켜서 데이터의 수를 인위적으로 늘리는** 방식입니다.
 - **변형 종류:** 영상 이동, 회전, 확대/축소, 대칭 이동(좌우 반전) 등 다양한 기법을 사용할 수 있습니다.
 - **Keras 구현:** `ImageDataGenerator` 함수를 사용하여 데이터를 증강합니다.

5.2. 대표적인 CNN 모델

- ***ILSVRC (ImageNet Large Scale Visual Recognition Challenge)***는 대규모 영상 데이터셋(ImageNet)을 사용하여 영상 분류 및 인식 성능을 겨루는 국제적인 경연 대회입니다. 이 대회에서 우승한 모델들은 CNN 발전의 중요한 이정표가 되었습니다.
- **LeNet-5 (1998년):**
 - **특징:** Yann LeCun에 의해 1998년에 제안된 **최초의 CNN 구조**로 평가됩니다.
 - **구조:** 32x32 픽셀의 손글씨 숫자를 인식하기 위해 Conv-Pool-Conv-Pool-FC-FC-FC의 **7개 계층**으로 구성되었습니다.
 - **활성화 함수:** Sigmoid 또는 tanh 활성화 함수를 사용했습니다 .

- **AlexNet (2012년):**
 - **특징:** ILSVRC 2012년 대회에서 **우승**하며 딥러닝의 가능성을 전 세계에 알린 모델입니다.
 - **구조:** 총 **8개의 레이어**로 구성되었으며, 2개의 GPU를 사용하여 병렬 연산을 수행하도록 설계되었습니다.
 - **훈련 방식:** **최초로 ReLU를 활성화 함수로 사용**했으며, Local Response Normalization, Dropout 0.5, Data Augmentation, 배치 크기 128 등의 훈련 방식을 적용했습니다.
 - **파라미터 수:** 총 62,378,344개(약 6,200만 개)의 파라미터를 가집니다 .
- **VGGNet (2014년):**
 - **특징:** ILSVRC 2014년 대회에서 **우승**한 모델입니다.
 - **깊이의 중요성:** 깊이를 늘리는 것이 중요함을 보여주었으며, 비교적 작은 3x3 필터와 최대 풀링 계층을 반복적으로 쌓는 단순한 구조를 가집니다.
 - **파라미터 수:** 총 138M(1억 3천 8백만)개의 파라미터를 가집니다.
 - **Cifar-10 적용:** VGGNet 구조를 32x32 Cifar-10 영상에 적용할 경우, 10개 클래스 출력을 위해 총 **12개의 계층**으로 구성될 수 있으며, 약 75%의 정확도를 보입니다.

6. 전이 학습 (Transfer Learning)

전이 학습은 이미 대규모 데이터셋(예: ImageNet)으로 학습되어 특정 작업을 수행할 수 있는 신경망 모델의 **지식과 특징 추출 능력**을 새로운, 관련 있는 문제에 **전달하여 활용**하는 강력한 기술입니다.

- **개념:** 기존에 잘 학습된 신경망 모델에서 ****특징 추출 부분(하위 계층)****은 그대로 사용하고, 이 신경망의 ****최상위 부분(출력 계층)****만 새로운 작업에 맞게 새로 정의한 다음, 이 새로 정의된 부분만 훈련시키는 것을 '전이 학습'이라고 합니다.
- **장점:**
 - **적은 데이터로 학습:** 하위 부분은 이미 학습된 신경망을 사용하기 때문에, 새로운 문제에 대해 **적은 양의 데이터만으로도 시스템을 효율적으로 학습**시킬 수 있습니다. 이는 처음부터 모델을 학습시키는 것보다 훨씬 빠르고 비용 효율적입니다.
 - **성능 향상:** 이미 대규모 데이터로 일반적인 특징을 잘 학습한 모델을 기반으로 하므로, 새로운 작업에서도 더 높은 성능을 달성할 가능성이 높습니다.
- **Keras CNN 애플리케이션 (Applications):**
 - Keras는 `tensorflow.keras.applications` 모듈을 통해 ImageNet 데이터로 미리 훈련된 ResNet50, VGG16 등과 같은 강력한 신경망 모델들을 **쉽게 불러와 사용할 수 있도록** 제공합니다.
 - 이 모델들은 분류기로서 최종 예측에 사용될 수도 있고, `include_top=False` 옵션을 통해 최종 예측 계층을 제외하고 **특징 추출기**로만 활용될 수도 있습니다. 중간 계층의 결과도 추출할 수 있습니다 .

6.1. VGG16을 이용한 개-고양이 인식 예시 (전이 학습)

제공된 자료에는 VGG16 모델을 전이 학습 방식으로 사용하여 개와 고양이를 분류하는 예시가 제시되어 있습니다.

- **기본 아이디어:** 이미 ImageNet으로 훈련된 VGG16 모델의 CNN 부분(특징 추출기)은 고정하고, 개와 고양이 분류를 위한 최상위 부분(분류기)만 새로 정의하여 훈련합니다.
- **데이터 준비:**
 - Kaggle의 'dogs-vs-cats' 데이터셋(25,000장)에서 2,000장의 서브셋(subset)을 사용합니다 .

- 영상의 크기를 150x150 픽셀로 조정합니다.
 - `ImageDataGenerator` 함수를 사용하여 훈련, 검증, 테스트 디렉토리에서 영상 데이터를 읽어옵니다.
- **특징 추출:** `conv_base.predict()` 함수를 이용하여 VGG16 모델(최상위 레이어 제외)로 각 영상의 특징을 추출합니다. 추출된 특징은 4x4x512 크기의 데이터로, 이를 1차원(8192 크기)으로 평탄화(Flatten)하여 최종 분류 계층에 전달합니다.
- **최종단 정의 및 훈련:**
 - 평탄화된 특징(8192차원)을 입력으로 받는 `Dense` (완전 연결) 계층을 새로 정의합니다.
 - `Dropout` 을 추가하여 과적합을 방지합니다 .
 - 이 새로 정의된 **최종단만** 훈련시킵니다.
- **장점:** 최종단만 훈련시키므로 전체 모델을 처음부터 훈련시키는 것보다 **수행 속도가 매우 빠릅니다**.
- **결과 확인:** 훈련 과정의 정확도(accuracy)와 손실(loss) 값을 그래프로 시각화하여 모델의 성능을 확인할 수 있습니다 .

9장 A 인식 - Detection

1. 인식 관련 문제: 인공지능이 '보는' 방법

인식은 컴퓨터가 영상이나 이미지 속의 내용을 이해하고 해석하는 과정을 말합니다. 여기에는 크게 세 가지 주요 문제가 있습니다.

1.1. 분류 (Classification)

컴퓨터가 영상에 있는 물체의 '종류'를 알아내는 문제입니다. 예를 들어, 주어진 사진이 '고양이'인지 '자전거'인지를 판단하는 것입니다.

- **부류 확률 벡터 출력:** 분류 결과로 해당 물체가 특정 부류에 속할 확률을 벡터 형태로 출력합니다.
- **사레 분류 (Instance Classification):** '내 차'나 '어린이 교통표지판'과 같이 특정 개체를 식별하는 것입니다.
- **범주 분류 (Categorical Classification):** '고양이'나 '자전거'처럼 일반적인 물체 부류를 식별하는 것입니다.
- **예시:** COCO 데이터셋은 사람, 자전거, 자동차 등 80가지의 부류를 다룹니다.

1.2. 검출 (Detection)

영상에서 물체의 '위치'를 찾아내고, 해당 물체가 무엇인지(부류)도 함께 알려주는 문제입니다.

- **바운딩 박스 (Bounding Box):** 물체의 위치를 직사각형 형태로 표시합니다.
- **미리 정해진 부류:** 보통 탐지할 부류는 미리 정해져 있습니다.

1.3. 분할 (Segmentation)

물체가 영상에서 차지하는 '정확한 영역'을 화소(픽셀) 단위로 지정하는 문제입니다.

- **화소 집합 지정:** 물체의 윤곽선을 따라 해당 물체를 구성하는 모든 화소들을 표시합니다.
- **의미 분할 (Semantic Segmentation):** 특정 부류에 속하는 모든 영역을 하나의 색으로 칠하는 방식입니다.
- **사레 분할 (Instance Segmentation):** 같은 부류에 속하더라도 개별적인 물체마다 다른 색으로 구분하여 표시하는 방식입니다.

2. 인식 데이터셋: 인공지능 학습을 위한 자료

인공지능 모델을 훈련하고 평가하기 위해서는 대규모의 영상 데이터셋이 필요합니다.

- **PASCAL VOC:** 20가지 부류에 대한 50만 장의 영상으로 구성되어 있습니다.

- **ImageNet:** 21,841가지 부류에 걸쳐 1,400만 장의 방대한 영상을 포함합니다.
 - ILSVRC 대회에서는 1,000부류를 선정하여 100만 장의 훈련 데이터, 5만 장의 검증 데이터, 15만 장의 테스트 데이터를 사용했습니다.
- **COCO:** 검출, 분할, 설명문 달기(captioning)에 사용되는 데이터셋으로, 80가지 부류에 대한 33만 장의 영상이 있습니다.
- **특정 분야 데이터셋:**
 - **DeepFashion:** 의류에 특화된 데이터셋으로, 50부류에 대한 80만 장의 영상이 있습니다.
 - **Food-101:** 101가지 음식 부류에 대한 약 10만 장의 영상이 있습니다.
 - **CheXpert:** 폐 엑스레이 사진 데이터셋으로, 65,240명의 환자에 대한 224,316장의 사진을 포함합니다.

3. 검출 (Object Detection): 물체 위치와 종류 찾아내기

검출은 영상에서 물체를 찾아 직사각형(바운딩 박스)으로 위치를 표시하고, 해당 물체의 부류 신뢰도를 나타내는 확률 벡터를 함께 출력하는 과정입니다. 딥러닝 기술의 발전으로 검출 성능이 획기적으로 개선되었으며, 자율주행차는 그 대표적인 성공 사례입니다.

3.1. 성능 척도 (Performance Metrics)

검출 모델의 성능을 평가하는 데 주로 mAP(mean Average Precision)라는 척도가 사용됩니다.

- **mAP (mean Average Precision):** 각 물체 부류에 대해 AP(Average Precision)를 구하고, 모든 부류에 대해 그 평균을 낸 값입니다.
- **AP (Average Precision) 계산:** IoU(Intersection over Union)를 기반으로 계산됩니다.
 - **IoU (Intersection over Union):** 두 개의 바운딩 박스(예측 박스와 실제 정답 박스)가 얼마나 겹치는지를 나타내는 척도입니다. (두 박스의 합집합 면적 대비 교집합 면적의 비율)
 - **AP 계산 알고리즘:**
 1. 모델이 예측한 박스들 중 신뢰도(confidence score)가 특정 임계값을 넘는 박스만 선택합니다.
 2. 선택된 예측 박스들을 신뢰도가 높은 순서대로 정렬하여 처리합니다.
 3. 각 예측 박스를 실제 정답 박스(참값 박스)와 비교하여 IoU를 계산합니다.
 - 만약 예측 박스와 참값 박스의 IoU가 미리 정해진 IoU 임계값(예: 0.5)을 넘으면, 해당 예측을 ****참 긍정(TP: True Positive)****으로 판단합니다. (만약 하나의 예측 박스가 여러 참값 박스와 겹칠 경우, IoU가 가장 큰 참값 박스와 짝을 이룹니다).
 - IoU 임계값을 넘지 못하면 ****거짓 긍정(FP: False Positive)****으로 판단합니다.
 4. 참 긍정으로 판정된 경우, 해당 참값 박스는 다시 사용되지 않도록 목록에서 제거합니다 (이중으로 짝을 맺는 것을 방지).
 5. 모든 예측 박스를 처리한 후에도 짝을 찾지 못한 참값 박스들은 ****거짓 부정(FN: False Negative)****으로 판단합니다.
 6. 이렇게 구해진 TP, FP, FN 값을 이용하여 다음 두 가지 값을 계산합니다:
 - **정밀도 (Precision):** $\frac{TP}{TP + FP}$ (모델이 '맞다고 한 것' 중에서 실제 정답의 비율)
 - **재현률 (Recall):** $\frac{TP}{TP + FN}$ (실제 정답 중에서 모델이 '맞다고 찾아낸 것'의 비율)
 7. 신뢰도 임계값을 0.0부터 1.0까지 점진적으로 변화시키면서 해당 정밀도와 재현률을 측정합니다. 이 값들을 가로축(재현률)과 세로축(정밀도)에 배치하여 그래프를 그린 것이 '정밀도-재현률 그래프'입니다. AP는 이 그래프 아래 면적을 의미합니다.

3.2. 사물 영역 탐지에서의 문제점

실제 영상에서 사물을 탐지하는 것은 다음과 같은 어려움이 있습니다 :

- **다중 사물:** 영상 안에 몇 개의 사물이 있을지 알 수 없습니다 .
- **사물 종류의 다양성:** 탐지해야 할 사물의 종류가 매우 다양할 수 있습니다 .
- **영상 크기의 변화:** 입력 영상의 크기가 다양하게 변할 수 있습니다 .

3.3. 다중 사물 영역 탐지 기법

입력 영상에는 일반적으로 여러 개의 사물 영역이 있습니다 .

- **Sliding Window (슬라이딩 윈도우) 방식:**
 - 영상의 많은 부분 영역에 대해 CNN(Convolutional Neural Network)을 적용하여 각 영역이 사물인지 배경인지를 판단하는 초기 방법입니다 .
 - **문제점:** 영상 내에 몇 개의 사물이 있을지 알 수 없고, 사물 영역의 수가 매우 많을 수 있어 모든 영역에 대해 CNN을 적용하는 것은 매우 비효율적입니다 .
- **영역 제안 (Region Proposals):**
 - 모든 물체 영역을 포함할 수 있는 가능한 한 적은 수의 후보 상자(region proposals)를 찾는 방법입니다 .
 - 일반적으로 발견적(heuristic) 탐색 기법을 사용하여 2,000개 정도의 후보 영역을 탐색합니다 .
- **중복된 탐지 영역 제거 (Non-max Suppression, NMS):**
 - 사물 탐지 결과에는 서로 겹쳐진 예측 박스들이 많이 나타납니다 .
 - NMS는 이러한 중복된 영역들을 제거하는 기법입니다 .
 - **작동 방식:** IoU가 일정한 값(예: 0.7) 이상인 박스들을 중첩 영역으로 간주하고, 이들 중 신뢰도가 낮은 박스를 제거하는 방식으로 중복을 줄여 나갑니다 .

4. 주요 사물 탐지 기술: 딥러닝의 발전

사물 탐지 기술은 CNN 기반 영상 인식 기술을 활용하여 발전해 왔습니다 .

4.1. R-CNN (Region-based CNN, 2014)

R-CNN은 CNN 기반의 초기 사물 탐지 기술로, 2014년에 발표되었습니다 .

- **핵심 아이디어:**
 - 영상에서 약 2,000개 정도의 '영역 제안(region proposals)'을 미리 찾아냅니다 .
 - 각각의 제안된 영역에 대해 별도로 CNN 분석을 수행하여 물체 분류 및 위치를 조정합니다 .
- **구성 요소 :**
 - **영역 제안 모듈:** '선택적 탐색(selective search)' 방식을 사용하여 관심 영역(Rol: Regions of Interest)을 추출합니다 .
 - **선택적 탐색 절차 :** 영상에서 초기 영역을 설정하고, 유사한 인접 영역들을 NMS를 이용하여 통합하여 후보 영역을 생성합니다.
 - **특징 추출 모듈:** 사전 학습된 CNN을 이용하여 각 Rol의 특징을 추출합니다 .
 - **분류 모듈:** SVM(Support Vector Machine)을 이용하여 해당 영역에 어떤 물체가 있는지 분류합니다 .
 - **위치 특징 모듈 (Bounding-box Regressor):** 물체의 경계 박스(x, y, 너비, 높이)를 더 정확하게 조정합니다 .
- **학습 과정 :**
 - Rol 추출(선택적 탐색)은 기존 알고리즘을 사용하고, 나머지 모듈들을 개별적으로 학습시킵니다 .
 - **특징 추출 모듈:** 기존 CNN 모델을 미세 조정하여 사용합니다 .

- SVM 분류기: 각 물체 종류에 대한 레이블 데이터를 학습시킵니다 .
- 경계 박스 회귀 모듈: 경계 박스 데이터를 이용하여 학습시킵니다 .

- **단점 :**
 - **느린 속도:** 영상 한 장마다 2,000개 이상의 Rol에 대해 개별적으로 CNN을 적용하므로 계산량이 매우 많습니다. 실시간 시스템에는 적합하지 않습니다 .
 - **비효율적인 CNN 적용:** CNN이 각 영역별로 따로 적용되어 비효율적입니다 .
 - **복잡한 학습 과정:** 학습이 여러 단계로 구성되어 있습니다 .
 - **높은 자원 요구량:** 추출된 특징을 저장하기 위해 수백 GB의 저장 공간이 필요할 정도로 학습의 공간 및 시간 복잡도가 높습니다 .

4.2. Fast R-CNN

Fast R-CNN은 R-CNN의 느린 속도 문제를 개선하기 위해 제안되었습니다 .

- **핵심 개선:** R-CNN처럼 각 Rol마다 CNN을 적용하는 대신, **전체 영상에 대해 CNN을 먼저 한 번만 적용**한 다음, 이 특징 맵(feature map)에서 Rol에 해당하는 부분을 추출하여 분석합니다 .
- **특징 :**
 - 특징 추출기, 물체 분류기, 경계 박스 회귀 모듈이 모두 하나의 CNN 모델에 포함되어 있어 R-CNN보다 수행 속도가 빠릅니다 .
 - 전체 영상에 대한 CNN이 한 번만 수행되므로 효율적입니다 .
 - SVM 분류기 대신 소프트맥스(softmax) 층을 사용하여 분류를 수행합니다 .
- **한계:** 여전히 '영역 제안'을 생성하는 데 사용되는 '선택적 탐색' 알고리즘이 느리다는 한계가 있었습니다 .

4.3. Faster R-CNN

Faster R-CNN은 Fast R-CNN의 '느린 영역 제안' 문제를 해결하기 위해 개발되었습니다 .

- **핵심 개선:** 느린 선택적 탐색 대신 ****RPN(Region Proposal Network)****이라는 신경망을 사용하여 영역 제안을 자동으로 검출합니다 . 이 RPN도 훈련 데이터를 통해 학습됩니다 .
- **Region Proposal Network (RPN) 상세 :**
 - 입력 영상을 CNN으로 처리한 후, 이 CNN의 출력(특징 맵)에 RPN을 적용합니다 .
 - 특징 맵의 각 위치(이를 'feature point'라고 합니다)를 중심으로 여러 개의 가상적인 '앵커 박스(anchor box)'를 설정합니다 .
 - 각 앵커 박스가 '사물 영역(positive)'인지 '배경(negative)'인지를 분류합니다 .
 - 앵커 박스에 대한 분류 학습은 실제 사물 영역이 표시된 훈련 영상들을 이용합니다 .
 - 사물 영역의 크기와 위치를 더 잘 맞추기 위해 '변환(transform)'을 사용하여 앵커 박스를 조정할 수 있습니다 .
 - 성능 개선을 위해 각 위치에서 다양한 크기와 비율을 가진 여러 개의 앵커 박스(예: 6개)를 사용합니다 .
 - 이 과정을 통해 각 영상에서 수많은 사물 영역이 생성되며, 이들을 정렬하여 최종적으로 300개 정도의 사물 영역을 추천합니다 .
- **수행 방식:** Faster R-CNN은 RPN 단계와 Fast R-CNN 단계, 두 단계로 구성됩니다 . 먼저 RPN을 수행하여 영역 제안을 얻고, 그 다음 이 영역 제안들에 대해 Fast R-CNN을 수행합니다 . 이로써 전체 과정이 매우 빨라졌습니다.

4.4. YOLO (You Only Look Once)

YOLO는 Faster R-CNN 이후에 등장한 기술로, 속도 개선에 중점을 둡니다 . GoogleNet과 같은 영상 인식 기술을 기반으로 합니다 .

- **핵심 특징:**

- R-CNN 계열보다 정확률은 다소 낮을 수 있지만, **월등히 빠릅니다** .
- 물체의 위치와 부류 정보를 '회귀(regression)' 방식으로 **한 번에(single-shot)** 알아내는 '한 단계(one-stage)' 방법'입니다 . 즉, 영역 제안과 분류를 분리하지 않고 동시에 처리합니다.
- **처리 과정** :
 1. 입력 영상을 S*S 격자(grid)로 나눕니다 (예: S=7) .
 2. 각 격자 칸은 해당 칸에 물체의 중심이 위치하는 경우 그 물체를 탐지할 '책임'을 가집니다 .
 3. 각 격자 칸은 여러 개의 바운딩 박스를 예측하며, 각 박스에 대해 위치 정보(x, y, 너비, 높이)와 해당 박스에 사물이 있을 신뢰도(o)를 출력합니다 .
 4. 또한 각 칸은 미리 정해진 물체 부류에 대한 확률(예: COCO 데이터셋에서는 P1~P80까지 80가지 부류 확률)을 출력합니다 .
 5. 결과적으로 각 칸은 이러한 정보들을 포함하는 90차원 벡터(예: 7*7=49개 칸, 한 칸당 90차원 벡터)로 표현됩니다 .
- **버전**: YOLO는 지속적으로 개선되어 여러 버전이 존재합니다 .

9장 B 인식 - Segmentation

1. 영상 분할이란 무엇인가요?

****영상 분할(Image Segmentation)****은 딥러닝 기술을 활용하여 이미지의 각 픽셀에 특정 의미를 부여하거나 객체를 분리하는 기술을 말합니다.

이는 마치 그림의 모든 점(픽셀)에 이름을 붙여주는 작업과 같습니다.

이 과정은 '밀집 분류(dense classification)' 문제로 간주될 수 있는데, 이는 이미지 내의 모든 픽셀마다 어떤 부류(카테고리)에 속하는지 분류하는 것이기 때문입니다.

2. 영상 분할의 주요 종류

영상 분할은 크게 세 가지 유형으로 나눌 수 있으며, 각각의 목적과 결과가 다릅니다.

- **의미 분할 (Semantic Segmentation)**
 - **목적**: 이미지 내의 **모든 화소(픽셀)에 특정 '부류(category)'를 할당**하는 것입니다.
 - **특징**: 예를 들어, 이미지에 여러 대의 자동차가 있다면, 모든 자동차 픽셀을 하나의 '자동차' 부류로 분류합니다. 이 방식은 **같은 부류 내의 개별 객체(인스턴스)는 서로 구분하지 않습니다**.
 - **분할 대상**: 'thing(개체)'과 'stuff(배경)' 모두를 분할합니다. 즉, 사람, 자동차와 같은 구체적인 '개체'뿐만 아니라 도로, 하늘과 같은 '배경' 영역까지 모든 픽셀에 레이블을 부여합니다.
 - **훈련 방식**: 영상의 각 화소에 해당하는 카테고리 레이블을 부여하고 시스템을 훈련시킵니다.
- **사레 분할 (Instance Segmentation)**
 - **목적**: 이미지 내의 '**물체(객체)**'에 **초점을 맞춰 분할**합니다.
 - **특징**: 의미 분할과 달리, **같은 부류의 물체가 여럿 있을 경우, 각각의 개별 물체에 번호를 붙여 구분**합니다. 예를 들어, 여러 대의 자동차가 있다면 '자동차 1', '자동차 2' 등으로 각각을 구별하여 분할합니다.
- **총괄 분할 (Panoptic Segmentation)**
 - **목적**: **의미 분할과 사레 분할의 장점을 결합**한 형태입니다.

- **특징**: 이미지의 **모든 화소에 부류를 할당**하는 동시에, **같은 부류의 물체가 여럿 있으면 번호를 붙여 구분**합니다. 이는 모든 픽셀을 분류하면서 동시에 개별 객체를 식별해야 하는 가장 포괄적인 분할 방식입니다.

3. 영상 분할 모델의 성능은 어떻게 측정하나요?

영상 분할 모델이 얼마나 정확하게 픽셀을 분류하는지 평가하기 위해 다음과 같은 척도들이 주로 사용됩니다.

- **화소 정확률 (Pixel Accuracy, PA)**
 - 모델이 예측한 픽셀 중 **전체 픽셀 대비 올바르게 분류된 픽셀의 비율**을 나타냅니다.
 - 분할 문제를 '화소마다 부류를 지정하는 문제'로 간주하여 성능을 측정합니다.
- **평균 화소 정확률 (Mean Pixel Accuracy, MPA)**
 - 각 부류(카테고리)별로 화소 정확률(PA)을 계산한 후, 이 값들의 **평균**을 낸 것입니다. 특정 부류에 대한 성능 편향을 줄일 수 있습니다.
- **IoU (Intersection over Union) 기반 척도**
 - **개념**: 예측된 분할 영역과 실제 정답 영역 간의 '**교집합(Intersection)**'을 '**합집합(Union)**'으로 나눈 값입니다. 이 값은 0과 1 사이이며, 1에 가까울수록 예측이 정답과 일치함을 의미합니다.
 - **활용**: 분할 품질을 평가하는 데 널리 사용되며, 객체 검출 모델에서 사용하는 AP(Average Precision) 및 mPA(mean Average Precision)와 유사한 개념을 사용합니다.

4. FCN (Fully Convolutional Networks): 영상 분할의 시작점

****FCN (Fully Convolutional Networks)****은 딥러닝을 활용한 ****의미 분할(Semantic Segmentation)**을 '처음 시도한 성공적인 컨볼루션 신경망'***입니다. 이 모델의 등장은 영상 분할 연구에 큰 전환점이 되었습니다.

FCN 등장 이전의 문제점

이전의 AlexNet, VGGNet 같은 일반적인 CNN(Convolutional Neural Network) 모델들은 이미지 분류를 위해 네트워크의 마지막 부분에 '밀집 연결층(Dense net)'을 가지고 있었습니다. 이 밀집 연결층은 전체 이미지에 대한 하나의 분류 결과를 도출하는 데는 효과적이었지만, 각 픽셀의 **위치 정보나 픽셀 간의 연결 관계를 잃어버리게** 하는 문제가 있었습니다. 이로 인해 픽셀 단위로 세밀하게 분류해야 하는 영상 분할 작업에는 적합하지 않았었습니다.

FCN의 혁신

FCN은 이러한 문제점을 해결하기 위해 네트워크 전체를 ****컨볼루션층만으로 구성****했습니다. 이를 그대로 'Fully Convolutional'한 특징을 가짐으로써, 픽셀의 위치 정보 손실 없이 각 픽셀에 대한 분류 결과를 효과적으로 도출할 수 있게 되었습니다. FCN의 성공 이후, 다양한 아이디어로 개선된 모델들이 여럿 등장하여 영상 분할 기술 발전에 기여했습니다.

4.1 FCN의 구조와 동작 원리

FCN은 크게 두 가지 과정으로 구성됩니다.

- **기본 구조**:
 - 일반적인 CNN에서 특징 추출을 위해 이미지 크기를 줄여나가는 '**다운샘플링(downsampling)**' 과정과,
 - 추출된 특징 맵의 크기를 다시 원래 이미지 크기로 복원하는 '**전치 컨볼루션(transposed convolution)**'을 이용한 '**업샘플링(upsampling)**' 과정입니다.
- **입력 및 출력**:
 - **입력**: 일반적인 m x n 크기의 3채널 컬러 영상(가로 m, 세로 n, RGB 3채널)입니다.

- 출력: 입력 영상과 동일한 크기(m x m)의 '맵(map)' 형태로 나옵니다. 이 출력 맵의 각 픽셀은 (C+1)차원의 '부류 확률 벡터'를 가집니다. 여기서 'C'는 분류하고자 하는 부류의 개수를 의미하며, '+1'은 '배경' 부류를 포함합니다.
- 출력층 처리:
 - FCN의 출력층은 각 화소(픽셀)별로 'softmax' 함수를 적용하여 (C+1)차원의 부류 확률 벡터를 생성합니다. 이 벡터는 각 픽셀이 어떤 부류에 속할 확률을 나타냅니다.
 - 모델이 학습할 때 참조하는 올바른 정답 데이터인 '참값(ground truth)'은 각 화소별로 'one-hot code' (예: 배경은, 부류1은 등)로 표현됩니다.
- 다운샘플링과 업샘플링의 역할:
 - 다운샘플링: 입력 영상의 크기를 점진적으로 줄여나가면서 영상의 중요한 '고수준 특징(high-level features)'을 추출하는 과정입니다.
 - 업샘플링: 다운샘플링으로 크기가 줄어든 특징 맵을 다시 원래 영상 크기로 복원하여 픽셀 단위의 분할 결과를 만들기 위한 과정입니다. FCN은 이 업샘플링을 '전치 컨볼루션'이라는 독특한 방식을 통해 달성합니다.
 - 전통적인 이미지 확대 방식인 '양선형 보간법'을 사용할 경우, 다운샘플링은 모델이 학습하지만 업샘플링은 고정된 방식을 사용하는 셈이 됩니다. 하지만 FCN은 업샘플링마저도 학습 가능한 전치 컨볼루션을 사용하여 네트워크 전체가 최적화되도록 합니다.

4.2 전치 컨볼루션(Transposed Convolution) 상세 설명

전치 컨볼루션은 FCN에서 핵심적인 업샘플링 기술로, 이름처럼 '컨볼루션' 연산을 역으로 수행하여 특징 맵의 크기를 키우는 방식입니다. 간혹 'deconvolution'이라고도 불리지만, 'transposed convolution'이 일반적인 명칭입니다.

- 개념: 입력 맵의 크기를 '업샘플링(upsampling)'하기 위해 기존 '컨볼루션(Convolution)' 연산을 역방향으로 수행하는 것입니다.
- 동작 방식:
 - 0 삽입: 컨볼루션을 수행하기 전에, 입력 맵의 각 행과 열 사이에 보폭(stride) s에서 1을 뺀 만큼(s-1)의 0을 추가하여 맵의 크기를 인위적으로 늘립니다.
 - 컨볼루션 마스크 계산: 0이 추가된 맵에 일반적인 컨볼루션 마스크(필터)를 적용하여 계산을 수행합니다.
 - 행렬 연산: 수학적으로는 컨볼루션 연산을 행렬 곱셈으로 표현할 때, 그 '전치 행렬의 곱'으로 구현될 수 있습니다.
 - FCN에서는 이 전치 컨볼루션 마스크의 계수(필터 값)를 훈련 데이터를 통해 학습하여, 원래 이미지와 최대한 유사한 형태의 결과 이미지를 복원하도록 합니다.
- FCN에서의 활용: FCN은 이 전치 컨볼루션을 사용하여 다양한 크기(3단계 스케일)의 특징 맵을 결합함으로써 '거친 특징(coarse features)'과 '세밀한 특징(fine features)'을 모두 추출합니다. 최종적으로 (C+1)개의 1x1 필터(마스크)를 적용하여 각 부류별로 분할된 맵을 가진 텐서(데이터 묶음)로 변환합니다.

5. FCN을 개선한 모델들

FCN의 성공 이후, 이를 바탕으로 성능과 효율성을 더욱 높은 다양한 신경망들이 등장하여 영상 분할 기술 발전을 이끌었습니다.

- DeConvNet
 - 구조: '오토인코더(Autoencoder)'와 FCN을 결합한 형태의 신경망입니다. 오토인코더처럼 인코더(Encoder)는 다운샘플링을 통해 특징을 추출하고, 디코더(Decoder)는 업샘플링을 통해 분할 맵을 생성합니다.
 - 성능: FCN에 비해 학습이 훨씬 정교하며 성능도 우수하다고 평가됩니다.
 - 업샘플링: FCN과 마찬가지로 업샘플링에는 전치 컨볼루션을 적용합니다.
- U-Net
 - 개발 목적: 원래 '의료 영상 분할'을 목적으로 개발되었으나, 지금은 일반 자연 영상 분할에도 널리 사용되고 있습니다.

- 기본 구조: FCN과 유사하게 다운샘플링과 업샘플링 과정을 수행합니다.
- 가장 큰 특징: '지름길 연결(shortcut connection)' 또는 '스킵 연결(skip connection)'이라고 불리는 독특한 구조를 가집니다. 이 지름길 연결은 다운샘플링 과정에서 얻은 특징 맵(인코더의 결과)을 업샘플링 과정의 해당 단계(디코더)에 직접 연결(이어 붙임)하여, **고해상도 특징 정보가 손실되지 않고 최종 분할 결과에 반영**되도록 돕습니다.
- 활용 분야: U-Net은 생체 의료 영상 분할 분야에서 특히 많이 활용됩니다. (예: 도로 영상 분할 사례 등 다양한 분야에서 활용됩니다).

10장 동적 비전

1. 동적 비전이란 무엇인가?

1.1. 동적 비전의 정의

동적 비전은 고정된 카메라로 움직이는 물체를 관찰하거나, 움직이는 카메라로 영상을 얻는 기술을 의미합니다. 이는 시간에 따라 변화하는 영상 정보를 분석하여 물체의 움직임이나 속도 등을 파악하는 분야입니다.

1.2. 주요 활용 분야

동적 비전은 다양한 실제 문제 해결에 활용될 수 있습니다:

- 물체 추적(Object Tracking): 움직이는 물체의 경로를 따라갑니다.
- 속도 계산: 물체의 이동 속도를 측정합니다.
- 감시(Surveillance): 특정 지역에서의 이상 움직임을 감지하고 모니터링합니다.
- 자율주행: 차량 주변의 움직임을 감지하여 안전한 운행을 돕습니다.
- 환경 감시: 자연 환경의 변화나 특정 움직임을 추적합니다.
- 스포츠 분석: 운동선수나 공의 움직임을 분석하여 경기력을 향상시킵니다.

1.3. 응용 분야의 제한점

일부 능동 비전 방식(예: 라이다, 초음파 센서)이나 스테레오 비전은 3D 정보를 추출하는 데 유용하기 때문에, 동적 비전의 수동 모션 해석 기법은 응용 분야가 상대적으로 제한될 수 있다고 언급됩니다.

1.4. 비디오(동영상)의 이해

동적 비전의 기본이 되는 **비디오(동영상)**는 '시간 순서에 따라 여러 장의 정지 영상을 연속적으로 나열한 구조'를 말합니다.

2. 움직이는 부분을 찾는 방법: 차이 영상(Difference Image)

2.1. 차이 영상의 개념

차이 영상은 고정된 카메라로 촬영된 비디오에서 '움직이는 부분'만을 찾아내는 방법입니다. 이는 미리 정의된 '배경 영상'과 현재 영상을 비교하여 움직임을 검출하는 방식입니다. 마치 그림 찾기 놀이에서 다른 부분을 찾는 것과 비슷하다고 생각할 수 있습니다.

2.2. 차이 영상 계산 원리

차이 영상은 다음 단계를 통해 계산됩니다:

- 배경 영상 준비: 물체들이 없는 상태의 배경 영상 I_b 를 미리 준비합니다.
- 현재 영상과의 비교: 현재 촬영된 영상 $f_t(x, y)$ 와 배경 영상 $f_r(x, y)$ 의 차이를 계산합니다.

3. 움직임 판단: 이 차이가 미리 정해둔 임계값 (τ)보다 크면 움직임이 있는 픽셀(점)로 판단하여 1을 부여하고, 그렇지 않으면 0을 부여합니다. 이를 통해 움직이는 영역만 '흰색'으로 나타나는 이진 영상을 얻을 수 있습니다.

차이 영상 계산식:

$$d_t(x, y) = \begin{cases} 1, & \text{if } f_t(x, y) - f_r(x, y) > \tau \\ 0, & \text{otherwise} \end{cases}$$

- $f_r(x, y)$: 물체가 없는 배경 영상
- $f_t(x, y)$: 현재 시점 t의 영상
- τ : 움직임으로 간주할 최소 밝기 변화량 (임계값)
- $d_t(x, y)$: 계산된 차이 영상

4. 잡음 처리: 만약 영상에 잡음이 심한 경우, '지역 평균'과 같은 스무딩(smoothing) 기법을 적용하여 잡음을 줄일 수 있습니다.

2.3. 차이 영상 계산 절차 상세

움직이는 부분을 더 정확하게 검출하기 위한 추가 절차는 다음과 같습니다:

- 위에서 설명한 방식으로 차이 영상 $d_t(x, y)$ 를 계산합니다.
- 계산된 차이 영상에서 서로 연결되어 있는 영역들(connected areas)을 찾습니다. 이는 움직이는 물체가 여러 픽셀로 이루어져 있기 때문입니다.
- 크기가 너무 작은 연결 영역들은 잡음으로 간주하고 제거합니다.
- 마지막으로, 적절한 모폴로지 연산(morphological operations, 영상의 형태를 변형하는 연산)을 수행하여 움직이는 영역의 모양을 다듬거나 구멍을 메우는 등의 후처리를 진행합니다.

2.4. 차이 영상의 한계

차이 영상 기법은 간단하지만 몇 가지 제한점이 있습니다:

- 그림자 문제:** 움직이는 물체의 그림자도 함께 움직임으로 감지될 수 있습니다.
- 환경 변화에 취약:** 비, 눈과 같은 기후 변화나 햇빛 밝기 변화 등 외부 환경의 영향으로 인해 오차가 발생할 수 있습니다.
- 색상 변화:** 배경과 물체의 색상에 큰 변화가 없는 상황에서만 잘 동작합니다.

3. 물체의 움직임 추적: 광류(Optical Flow)

3.1. 광류의 개념

광류(Optical Flow)는 움직이는 물체의 '속도'를 추적하기 위해, 영상 내의 각 점(픽셀)이 다음 순간에 어디로 이동했는지를 나타내는 벡터(방향과 크기를 가진 값)를 계산하는 기술입니다. 쉽게 말해, 영상 속 모든 점들이 어떻게 움직이는지 화살표로 표현하는 것이라고 할 수 있습니다. 광류는 특징점(모서리 등)에 대해서만 계산하거나, 영상의 모든 영역에 대해 계산할 수 있습니다.

3.2. 광류의 종류: 특징점 vs. 전체 영역

3.2.1. 특징점에 대한 광류

- 대상:** 물체의 모서리처럼 특징적인 점들을 대상으로 합니다.
- 계산 방식:** '블록 매칭(block matching, SSD 등)'과 같은 방식을 사용하여 해당 특징점이 다음 영상에서 어디로 이동했는지 '대응점'을 찾습니다.
- 한계:** 모션 벡터(움직임 벡터)는 2D 영상 영역 내에서의 탐색이 필요하기 때문에, 3D 정보를 활용하는 스테레오 비전(두 대의 카메라로 깊이 정보를 얻는 방식)에 비해 정확도가 떨어질 수 있습니다.

3.2.2. Dense Optical Flow (전체 영역 광류)

- 대상:** 영상의 '모든 점(픽셀)'에 대한 광류를 계산합니다.
- 계산 방식:** 반복적인 최적화 기법을 사용하여 찾아낼 수 있습니다.

밝기 항상성(Brightness Constancy) 가정

Dense Optical Flow 계산의 핵심 가정 중 하나는 ****밝기 항상성(Color constancy)****입니다. 이는 "물체의 같은 점은 비디오에서 같은 밝기를 가진다"는 가정입니다. 즉, 특정 물체 표면의 한 점은 움직이더라도 그 점의 밝기는 변하지 않는다고 보는 것입니다.

광류 계산식 유도

밝기 항상성 가정을 바탕으로 영상의 밝기 함수 $f(x, y, t)$ 에 대한 '테일러 전개'를 사용하면 다음과 같은 광류 방정식을 유도할 수 있습니다.

영상 밝기 함수 $f(x, y, t)$ 는 픽셀 (x, y) 의 시점 t 에서의 밝기 값을 나타냅니다.

시간

dt 후 픽셀이 $(x + dx, y + dy)$ 로 이동했다고 가정하면, 밝기 항상성 가정에 따라 $f(x + dx, y + dy, t + dt) \approx f(x, y, t)$ 가 성립합니다.

테일러 전개와 밝기 항상성 가정을 적용하면 다음과 같은 광류 방정식을 얻을 수 있습니다:

$$\frac{\partial f}{\partial x} \cdot u + \frac{\partial f}{\partial y} \cdot v + \frac{\partial f}{\partial t} = 0$$

- $u = dx/dt$: 픽셀의 x축 방향 속도
- $v = dy/dt$: 픽셀의 y축 방향 속도
- $\partial f / \partial x$: x축 방향으로의 밝기 변화 (x 방향 미분값)
- $\partial f / \partial y$: y축 방향으로의 밝기 변화 (y 방향 미분값)
- $\partial f / \partial t$: 시간 변화에 따른 밝기 변화 (시간 미분값)

이 식을 통해 모션 벡터 (u, v) 를 구할 수 있습니다.

3.3. 광류 계산의 문제점과 해결 방법

3.3.1. 문제점

광류 방정식은 각 픽셀마다 모션 벡터 (u, v) 를 구해야 하는데, 하나의 방정식에 미지수가 두 개(u, v)이므로 이 식만으로는 정확한 u, v 값을 바로 구할 수 없습니다.

3.3.2. 해결을 위한 추가 가정

이 '미지수 부족' 문제를 해결하기 위해 추가적인 가정이 필요하며, 대표적인 방법은 다음과 같습니다:

- Lucas-Kanade 방법:** "인접한 픽셀들은 서로 유사한 모션 벡터를 가질 것이다"라는 **지역 조건**을 사용합니다. 즉, 작은 영역 내에서는 모든 픽셀이 거의 같은 방향으로 움직인다고 가정합니다.
- Horn-Schunck 방법:** "영상 전체에 걸쳐 모션 벡터는 천천히, 부드럽게 변해야 한다"는 **광역 조건**을 사용합니다. 이는 모션 벡터가 갑자기 크게 변하지 않고 연속적일 것이라는 가정입니다.

3.3.3. Smoothness 조건

Dense Optical Flow 계산 과정에서는 **Smoothness 조건**이 적용됩니다. 이는 "광류(optical flow)가 영상의 각 점에서 연속적으로 변한다"고 가정하며, "모션 벡터의 미분값 또한 연속적이다"라고 가정합니다.

$$\nabla u^2 + \nabla v^2 = (\frac{\partial u}{\partial x})^2 + (\frac{\partial u}{\partial y})^2 + (\frac{\partial v}{\partial x})^2 + (\frac{\partial v}{\partial y})^2$$

이러한 가정들을 바탕으로, 영상의 각 점에서 밝기 변화를 $\partial f / \partial x, \partial f / \partial y, \partial f / \partial t$ 을 계산하고 Smoothness 조건을 적용하여 특정 조건식을 최소화하는 (u, v)값을 찾아냅니다.

4. 최신 기술 동향: 딥러닝 기반 광류 추정

4.1. 참값 레이블링의 어려움

광류는 영상의 모든 픽셀마다 정확한 모션 벡터(참값, Ground Truth)를 지정해야 하기 때문에, **실제 영상에서 참값을 수동으로 레이블링하는 것이 매우 어렵습니다**. 이는 딥러닝 모델을 학습시키기 위한 데이터셋 구축에 큰 도전 과제가 됩니다.

4.2. 주요 데이터셋

이러한 어려움을 해결하기 위해 다양한 방식으로 구축된 데이터셋이 활용됩니다:

- KITTI 데이터셋**: 자율주행 연구를 위해 차량에 설치된 센서(카메라, 라이다 등)를 통해 실제 도로 환경에서 수집된 데이터셋입니다.
- Sintel 데이터셋**: 컴퓨터 그래픽(CG)으로 제작된 애니메이션 영상으로, 모든 픽셀의 정확한 움직임(참값)을 알 수 있어 레이블링이 용이합니다.
- Flying Chairs 데이터셋**: 실제 영상 위에 CG로 만든 '의자'를 인위적으로 추가하여, 의자의 움직임에 대한 참값을 알고 있는 방식으로 제작된 데이터셋입니다.

4.3. 딥러닝 적용 사례

최근에는 딥러닝 기술, 특히 ****컨볼루션 신경망(CNN)****을 광류 추정(Optical Flow Estimation)에 적용하는 연구가 활발히 진행되고 있습니다.

- 2015년부터 CNN을 광류 추정에 활용하기 시작했습니다.
- 2016년에는 영상 분할(segmentation) 알고리즘의 결과를 광류 추정에 활용하는 연구도 있었습니다.
- '최근에는 CNN을 사용하여 optical flow를 계산하기도 함'이라는 내용은 딥러닝이 광류 추정의 주요 기술로 자리 잡았음을 보여줍니다.

5. 광류의 실제 활용 사례

광류는 실제 산업 및 연구 분야에서 다양하게 활용됩니다. 특히 **자율 주행 분야에서 스테레오 비전(Stereo Vision) 기술과 함께 활용**되기도 합니다. 이는 차량 주변의 움직임을 정확하게 파악하고 예측하는 데 중요한 역할을 합니다.

동적 비전은 움직이는 물체 분석의 핵심 기술이며, 특히 광류는 물체의 속도 및 움직임 추적에 중요한 역할을 합니다. 광류는 '밝기 항상성'이라는 가정에 기반하며, 계산 과정의 불확실성을 해소하기 위해 Lucas-Kanade나 Horn-Schunck와 같은 추가 가정을 활용합니다.

최근 딥러닝 기술의 발전은 광류 추정의 정확도와 적용 범위를 크게 확장시키고 있으며, 자율주행과 같은 중요 분야에서 그 활용 가능성이 높게 평가되고 있습니다. 그러나 딥러닝 모델 개발에 있어 '참값 레이블링의 어려움'은 지속적인 도전 과제로 남아 있습니다.

12장 3차원 비전

1. 3차원 공간의 중요성과 특성

컴퓨터 비전 분야에서 3차원 공간을 이해하는 것은 매우 중요합니다. 우리가 사는 세상은 3차원이지만, 일반적인 카메라는 이 3차원 공간을 2차원 평면인 영상으로 투사하여 기록합니다. 이 과정에서 **시차, 투영, 음영, 하이라이트 효과** 등 다양한 현상이 발생하며, 결과적으로 영상에서 물체까지의 거리를 정확하게 측정하기가 어렵습니다.

하지만 자율주행과 같은 현대적인 애플리케이션에서는 3차원 공간에 대한 처리가 필수적입니다. 예를 들어, 자율주행 차량이 안전하게 운행하려면 주변 물체까지의 대략적인 거리를 추정하거나, 스테레오 비전(두 대의 카메라를 사용하여 거리 계산)과 같은 방법을 통해 정확한 거리를 계산하는 기술이 반드시 필요합니다.

2. 카메라 캘리브레이션: 3차원 정보 추출의 핵심

영상이 3차원 공간을 2차원으로 투사한 결과물이라는 점 때문에, 2차원 영상에서 원래의 3차원 공간 정보를 정확히 복원하려면 추가적인 정보가 필요합니다. 바로 이 추가 정보, 즉 카메라의 **초점 거리, 스케일 파라미터, 좌표 회전 및 천이(이동)** 등의 정보를 획득하는 과정을 ****카메라 캘리브레이션(Camera Calibration)****이라고 합니다. 카메라 캘리브레이션은 카메라가 외부 세계와 어떤 기하학적 관계를 가지는지 알아내는 작업이라고 할 수 있습니다.

2.1. 카메라 기하 구조: 썸 렌즈 광학 (Thin Lens Optics)

카메라 렌즈는 빛이 통과하는 방식에 따라 상(像)을 맺습니다. 일반적인 렌즈는 ****광학축(optic axis)****에 평행하게 들어온 광선을 ****초점(focal point)****으로 모으고, 초점을 지나온 광선은 광학축에 평행하게 진행하며, 렌즈 중심을 통과한 직선은 직진하는 특성을 가집니다. 렌즈는 한 물체점에서 나온 모든 광선이 정확히 동일한 위치에 모이도록 제작됩니다.

만약 상이 맺히는 위치가 카메라의 영상 평면(이미지 센서)과 정확히 일치하지 않으면, 영상이 ****흐려지는 현상(blurring 또는 out-of-focus)****이 발생합니다. 이러한 블러링을 줄이기 위해서는 빛이 통과하는 렌즈의 범위, 즉 ****조리개 크기(aperture size)****를 줄여야 합니다.

2.2. 조리개 제어 (Aperture Control)

조리개는 카메라로 들어오는 빛의 양을 조절하는 장치입니다. 영상의 밝기는 이 조리개 크기와 ****셔터 속도(빛을 통과시키는 시간)****에 의해 결정됩니다.

사진에서 ****심도(Depth of Field)****는 초점이 맞는 범위, 즉 선명하게 보이는 거리의 범위를 의미합니다. 렌즈의 빛 통과 범위(조리개)를 작게 할수록 (이때 f-number는 커집니다), 초점이 맞는 범위인 심도가 더 깊어지게 됩니다.

2.3. 핀홀 카메라 모델 (Pinhole Camera Model) & 원근 투영 (Perspective Projection)

핀홀 카메라 모델은 이상적인 카메라 모델로, 조리개 크기를 매우 작게(f-number를 매우 크게) 가정하여 렌즈 중심을 통해서만 빛이 들어온다고 단순화합니다. 또한, 물체까지의 거리가 초점 거리(f)보다 훨씬 커서 상이 초점(f)에서 맺힌다고 가정합니다.

원근 투영은 3차원 공간상의 점 $[X\ Y\ Z]$ 를 2차원 평면 (u, v) 로 변환하여 영상을 얻는 과정을 설명합니다. 이 투영 과정에서 가장 중요한 점은 ****거리 정보(Z축 정보)****가 사라진다는 것입니다. 따라서 2차원 영상의 점 (u, v) 로부터 원래의 3차원 점 (X, Y, Z) 를 복원하려면 추가적인 정보가 반드시 필요합니다.

2.4. 3차원 기하 변환

영상 속 피사체들은 대부분 3차원 공간 속에서 움직입니다. 따라서 카메라와 물체 사이의 공간적인 관계를 정확히 파악하기 위해서는 3차원 공간에서의 ****회전(Rotation)****과 **천이(Translation, 이동)** 변환이 필요합니다.

2차원에서의 기하 변환이 3x3 동차 행렬로 표현되는 것처럼, 3차원에서는 4x4 행렬로 확장됩니다. 특히 3차원에서의 회전 행렬을 직접 유도하는 것은 복잡하기 때문에, 회전 각도 대신 r_1 부터 r_9 와 같은 매개변수로 표현하기도 합니다.

2.5. 영상 생성 기하: 외부 행렬과 내부 행렬

카메라가 3차원 세계를 2차원 영상으로 변환하는 과정은 두 가지 주요 행렬로 설명됩니다.

- 세계(World) 좌표계**: 로봇이나 이동체 등이 놓인 기준 좌표계를 의미합니다. 세계 좌표계 상의 한 점 $p = (x_w, y_w, z_w)$ 는 카메라의 시점에서 본 좌표계, 즉 카메라 좌표계 (x_c, y_c, z_c) 로 변환되어야 합니다.
- 외부 행렬 (Extrinsic Matrix) $(R|t)$** : 세계 좌표계의 점을 카메라 좌표계로 변환할 때 사용되는 행렬입니다. 이 행렬은 세계 좌표계를 회전(R)시키고 이동(t)시키는 역할을 하며, 카메라가 외부 세계와 상호작용하는 기하학적 관계를 표현하기 때문에

'외부 행렬'이라고 불립니다.

- **내부 행렬 (Intrinsic Matrix) K** : 카메라 좌표계의 3차원 점 (x_c, y_c, z_c) 를 2차원 영상 평면의 점 (u, v) 로 투영하는 데 사용됩니다. 이 행렬은 카메라의 초점 거리, 픽셀 크기 등 카메라 자체의 내부적인 동작 및 특성을 표현하기 때문에 '내부 행렬'이라고 불립니다.

결론적으로, 외부 행렬과 내부 행렬을 모두 적용하면 세계 좌표계 상의 3차원 점 (x_w, y_w, z_w) 를 카메라 영상의 2차원 점 (u, v) 로 변환할 수 있습니다.

2.6. 카메라 캘리브레이션의 의미 및 절차

앞서 언급했듯이, **카메라 캘리브레이션**은 카메라가 외부 세계와 어떤 기하학적 관계를 가지는지 알아내는 작업입니다. 이는 결국 앞에서 설명한 외부 행렬 $(R|t)$ 와 내부 행렬 K 의 값을 알아내는 과정입니다. 특히 내부 행렬을 알아내는 작업을 **내부 카메라 캘리브레이션**, 외부 행렬을 알아내는 작업을 **외부 카메라 캘리브레이션**이라고 부릅니다.

카메라 캘리브레이션은 1970년대부터 활발히 연구되어 왔으며, **격자 패턴(chessboard pattern)** 이용, 물체의 기하학적 특성 (예: **소실점**) 이용, 카메라 위치를 고정하고 회전하며 영상을 분석하는 방법, 그리고 최근에는 **딥러닝(Deep Learning)** 방법 등 다양한 방식이 개발되었습니다.

이 중 **Zhang 방법**은 격자 패턴을 이용하는 방식으로, 영상 획득 상황을 제어할 수 있다면 가장 정확하고 편리하여 널리 사용되고 있습니다. Zhang 방법을 이용한 카메라 캘리브레이션의 일반적인 절차는 다음과 같습니다:

1. **격자 패턴 부착 및 3차원 점 수집**: 바둑판 모양의 격자 패턴을 벽이나 칠판과 같은 평평한 곳에 부착하고, 격자 종이의 왼쪽 위를 세계 좌표계의 원점으로 가정하여 각 교차점의 3차원 좌표를 수집합니다.
2. **여러 장의 영상 획득**: 카메라 또는 격자 패턴이 부착된 칠판을 조금씩 이동하거나 회전시키면서 다양한 각도에서 여러 장의 격자 패턴 영상을 획득합니다.
3. **대응 쌍 수집**: `findChessboardCorners` 와 같은 함수를 사용하여 획득한 영상에서 격자 교차점(2차원 점)을 자동으로 검출합니다. 이후 1단계에서 수집한 3차원 점과 검출된 2차원 점의 '대응 쌍'을 만듭니다.
4. **매개변수 추정**: `calibrateCamera` 와 같은 함수에 이 대응 쌍을 입력합니다. 이 함수는 ****렌즈 왜곡(Lens Distortion)****까지 고려하여 카메라의 내부 행렬을 구성하는 5개의 매개변수와 외부 행렬을 구성하는 6개의 매개변수를 최적으로 추정해 줍니다.

3. 깊이 추정: 물체까지의 거리 측정

3.1. 깊이 영상의 필요성

일반적으로 많이 사용하는 **RGB 영상**은 저렴하게 얻을 수 있고 다양한 시각 정보(색상, 질감 등)를 제공하지만, 조명 조건에 매우 취약하다는 단점이 있습니다. 예를 들어, 야간이나 강한 햇빛으로 인해 **하이라이트**가 발생하면 물체 인식이 실패할 가능성이 높습니다.

반면, 물체까지의 거리를 직접 기록한 영상인 ****깊이 영상(Depth Image)****을 RGB 영상과 함께 사용하면 이러한 조명 변화에 강인해질 수 있습니다. 특히 로봇이 주변 환경과 상호작용하고 협업하기 위해서는 정확한 거리 정보가 필수적입니다. 컴퓨터 비전에서는 물체까지의 거리를 ****깊이(depth)****라고 부르며, 이를 전문적으로 획득해주는 카메라를 ****깊이 카메라(Depth Camera)****라고 합니다.

3.2. 깊이 영상 획득 방법

깊이 영상을 획득하는 방법은 크게 두 가지로 나눌 수 있습니다.

- **수동적 방법**: 외부에서 에너지를 투사하지 않고, 기존의 빛(주변 조명)을 이용하여 깊이를 추정하는 방식입니다. 대표적인 예는 ****스테레오 비전(Stereo Vision)****입니다. 두 대 이상의 카메라를 사용하여 여러 장의 영상을 획득한 후, 이 영상들을 분석하여 깊이 정보를 추출합니다. 이는 사람이 두 눈을 사용하여 거리를 측정하는 원리와 매우 유사합니다.
- **능동적 방법**: 센서 자체가 장면에 어떤 무늬나 파동(레이저, 초음파 등)을 능동적으로 쏜 후, 되돌아오는 신호를 분석하여 깊이 정보를 추출하는 방식입니다.

3.3. 스테레오 비전

스테레오 비전은 두 대의 카메라를 나란히 설치하여 영상을 취득하는 방식입니다. 두 카메라가 촬영한 영상은 서로 비슷하지만, 물체와 카메라의 거리에 따라 약간의 시차(parallax)가 발생하여 다르게 보입니다. 사람의 두 눈과 비슷하게, 두 카메라의 시선을 평행하게 맞추어 두면 두 영상 간의 ****대응점(corresponding points)****을 찾는 문제가 훨씬 쉬워집니다.

- **에피폴라 기하 (Epipolar Geometry)**: 스테레오 영상에서 한 이미지에 있는 특정 점에 대응하는 다른 이미지의 점은 단순히 이미지 전체를 탐색할 필요 없이, ****에피폴라 선(epipolar line)****이라는 특정 선 위에 존재한다는 기하학적 원리입니다. 특히 스테레오 카메라를 서로 나란히 평행하게 배치하면, 에피폴라 선들의 y 좌표가 서로 같게 되어 대응점 찾기가 더욱 간편해집니다.
- **깊이 측정**: 에피폴라 기하를 통해 두 영상에서 동일한 물체에 해당하는 대응점 `x_left` 와 `x_right` 를 찾으면, 이들의 x좌표 차이인 **변위(disparity) $p \equiv x_{right} - x_{left}$** 를 이용하여 물체까지의 깊이를 추정할 수 있습니다. 대응점만 정확히 찾아낼 수 있다면 특정 수식을 통해 깊이 추정이 가능해집니다.
- **대응점 탐색 시 문제점**: 하지만 실제 영상에서는 노이즈나 복잡한 표면 때문에 대응점을 찾는 것이 쉽지 않습니다. 이때 발생하는 **아웃라이어(outlier) 문제**는 에피폴라 기하를 통해 해결할 수 있으며, 대응점이 듬성듬성하게 나타나는 **희소한(sparse) 문제**는 변위 공간 탐색 등을 통해 해결합니다.

3.4. 밀집(Dense) 깊이 영상

일반적인 스테레오 비전은 특징점 위주로 깊이를 추정하지만, **밀집 깊이 영상**은 특징점뿐만 아니라 영상 내의 ****모든 화소(pixel)****에 대해 깊이 정보를 추정하는 알고리즘을 말합니다. 이는 예를 들어 에피폴라 선(l) 상에서 SSD(Sum of Squared Differences)가 최소인 점을 탐색하는 방식으로 구현될 수 있습니다.

최근에는 **CNN (Convolutional Neural Network) 기반 스테레오** 방식이 활발히 연구되고 있습니다. 이 방식은 기존 영상 대신 컨볼루션 신경망을 통해 처리된 결과에서 두 영상의 유사도를 측정하여 깊이를 추정합니다. DISPNet-C와 같이 레이블을 깊이 맵으로 설정하고 U-net과 같은 의미 분할(semantic segmentation) 모델을 학습시켜 깊이 영상을 얻는 방식도 있습니다. 이러한 딥러닝 기반 방식은 실시간으로 고해상도 깊이 영상을 얻는 데 유리합니다 .

3.5. 능동 센서

능동 센서는 스스로 빛이나 파동을 장면에 투사한 후, 되돌아오는 신호를 분석하여 거리를 측정하는 방식입니다. 대표적으로 **구조광(Structured Light)** 방식과 **비행 시간(Time-of-flight, ToF)** 방식이 있습니다.

- **비행 시간 (Time-of-flight, ToF) 센서**: 이 센서는 광선이나 파동을 쏜 뒤, 그것이 물체에 반사되어 센서로 되돌아오는 데 걸린 시간을 측정하거나 파동의 위상 차이를 분석하여 거리로 변환합니다.
 - **Lidar (Light + radar)**: 레이저를 사용하여 거리를 측정하는 ToF 방식의 대표적인 센서입니다. 주로 자율주행 차량에 많이 사용됩니다 .
- **구조광 (Structured Light)**: 이 방식은 스테레오 비전에서 카메라 한 대를 **프로젝터**로 대체한 것과 유사합니다. 프로젝터는 미리 설계된 정교한 패턴(예: 점, 선, 격자 무늬)을 장면에 투사합니다. 이 패턴은 모든 점이 다른 점과 구별될 수 있는 고유한 특징을 가지도록 설계되어 있어, 카메라가 이 패턴을 촬영하면 대응점을 쉽게 찾을 수 있습니다. 이를 통해 깊이를 계산합니다.
 - **Kinect (키넥트)**: 마이크로소프트의 ****초기 키넥트(2010년 출시)****는 적외선 패턴을 투사하는 구조광 방식을 사용했습니다. 적외선은 사람 눈에 보이지 않기 때문에 사용자 경험을 방해하지 않습니다.
 - **주요 사양**: 480x640 해상도의 RGB 영상과 깊이 영상을 초당 30프레임으로 동시에 획득할 수 있었습니다. 깊이 측정 범위는 주로 실내 환경에 적합한 0.8m에서 3.0m 사이였습니다.
 - **주목받는 이유**: 키넥트는 컬러 영상과 깊이 영상을 동시에 분석하여 사람의 손, 몸, 얼굴을 추적할 수 있는 인터페이스(함수)를 **SDK (Software Development Kit)** 형태로 제공하여 큰 주목을 받았습니다.
 - **깊이 영상의 한계**: 키넥트의 깊이 영상은 전선처럼 가느다란 물체나 물체의 경계선에서 측정 실패가 발생하는 경향이 있었습니다. 이는 동작 인식에는 큰 문제가 없었지만, 정밀한 3차원 지도 제작이나 로봇 항해와 같은 용도에는 문제가 될 수 있어 추가적인 깊이 영상 전처리 연산이 필요했습니다.
- **상용 깊이 카메라**: 2010년의 초기 키넥트와 달리, 2019년에 나온 ****애저 키넥트(Azure Kinect)****는 ToF 방식을 채택하여 컴퓨터 비전 용도로 더 적합하게 진화했습니다. 깊이 카메라가 획득하는 깊이 채널(D 채널) 영상은 너무 가깝거나 멀리 있는

물체, 너무 얇은 물체, 또는 하이라이트가 발생한 물체 등에서 잡음(noise)이 심하게 발생할 수 있으므로, 다양한 전처리 알고리즘이 필요합니다.

3.6. 점 구름 (Point Cloud) 데이터

라이다(Lidar) 센서와 같이 레이저를 사용하는 능동 센서가 획득하는 데이터는 **점 구름(Point Cloud)** 영상이라고 불립니다. 라이다라는 레이저가 도달할 수 있는 곳의 깊이만을 측정하기 때문에, 일반적인 깊이 영상처럼 모든 화소에 깊이 값이 값이 있는 것이 아니라, 듬성듬성한(sparse) 깊이 영상을 생성합니다.

이러한 최소 깊이 영상은 3차원 좌표 (x, y, z) 로 표현되는 점들의 집합으로 이루어져 있으며, 이를 점 구름이라고 합니다. 점 구름은 일반적인 2차원 배열 형태의 이미지와는 다르기 때문에, 이를 처리하고 인식하기 위한 별도의 알고리즘이 필요합니다. 점 구름 데이터의 중요한 특징 중 하나는 **순열 불변(permutation-invariant)** 특성을 가진다는 것입니다. 즉, 점들의 순서를 바꾸어도 동일한 데이터로 간주됩니다.

13장 생성 비전

1. 비전 인공지능의 두 가지 축: 분별과 생성

우리는 매일 수많은 이미지를 보고, 그 속에서 정보를 얻습니다. 인공지능도 마찬가지입니다. 인공지능이 이미지를 '이해'하고 '창조'하는 방법을 배우는 분야가 바로 ****비전 인공지능(Vision AI)****입니다.

비전 인공지능은 크게 두 가지 접근 방식으로 나눌 수 있습니다.

1.1. 분별(Discriminative) 비전이란?

- 목표:** 주어진 영상에서 어떤 물체가 있는지 '인식'하고 '구별'하는 것입니다.
- 작동 방식:** 입력 영상과 해당 영상이 어떤 종류(클래스)인지에 대한 정보를 함께 학습합니다. 예를 들어, 개 사진을 보고 '이것은 개다'라고 분류하도록 배우는 것이죠.
- 학습 방법:** 주로 ****지도 학습(Supervised Learning)****이라는 방법을 사용합니다. 정답이 있는 데이터를 이용해 학습하는 방식입니다.
- 예시:** 얼굴 인식, 사물 분류, 자율주행차의 장애물 감지 등.

1.2. 생성(Generative) 비전이란?

- 목표:** 기존의 데이터를 학습하여 완전히 '새로운' 데이터를 '생성'하는 것입니다. 단순히 인식하는 것을 넘어, 세상에 없던 것을 만들어내는 창조적인 역할입니다.
- 작동 방식:** 특정 클래스(예: 고양이)에 속하는 영상들을 학습한 후, 실제로처럼 보이는 가짜(fake) 영상을 만들어냅니다.
- 예시:** 사람의 얼굴 생성, 가상 인물 생성, 특정 스타일의 그림 생성, 텍스트를 이용한 이미지 생성(DALL-E) 등.

2. 생성 모델이란 무엇일까요?

생성 비전의 핵심은 바로 ****생성 모델(Generative Model)****입니다. 생성 모델은 데이터가 어떤 모습으로 나타날지 '확률적'으로 이해하고, 이를 바탕으로 새로운 데이터를 만들어냅니다.

2.1. 생성 모델의 기본 개념

- 확률 분포 $p(x)$:** 생성 모델은 어떤 데이터(x)가 발생할 '확률 분포(probability distribution) $p(x)$ '를 학습합니다. 이 확률 분포를 알면, 우리는 그 분포에서 새로운 데이터 샘플을 무한히 생성해낼 수 있습니다.
- 비지도 학습(Unsupervised Learning):** 생성 모델은 데이터의 종류(레이블)를 알려주지 않아도, 오직 데이터 자체(x)만을 가지고 그 데이터의 확률 분포를 스스로 추정합니다. 그래서 '선생님(레이블)'이 없이 배우는 **비지도 학습**에 해당합니다.

- 예시: 주사위 눈금 생성:** 가장 간단한 예는 주사위 눈금을 생성하는 것입니다. 각 눈금(1부터 6까지)이 나올 확률이 1/6이라는 것을 알면, 이 확률 분포를 이용해 무작위로 주사위 눈금을 생성할 수 있습니다.

2.2. 다름 가능(Tractable) vs. 다름 불가능(Intractable)

생성 모델이 다루는 확률 분포는 그 복잡성에 따라 '다름 가능'하거나 '다름 불가능'으로 나뉩니다.

- 다름 가능 (Tractable):**
 - 쉬운 경우:** 키와 몸무게 데이터 생성처럼, 확률 분포 $p(x)$ 를 배열 형태로 표현하고 그 분포에서 샘플을 추출하는 것이 비교적 쉬운 경우입니다.
 - 이상적인 경우:** 만약 데이터셋 X 를 무한히 크게 만들 수 있다면, 모델이 추정한 확률 분포(p_{model})는 실제 데이터의 확률 분포(p_{data})와 거의 같아질 수 있습니다.
- 다름 불가능 (Intractable):**
 - 어려운 경우:** 실제 세상의 이미지 데이터는 수백에서 수만 개의 차원(픽셀)을 가지고 있고, 각 픽셀값도 수백 가지가 가능합니다. 이렇게 복잡한 데이터의 $p(x)$ 는 직접 표현하거나 학습하고, 거기서 샘플을 뽑아내는 것이 거의 불가능합니다.
 - 해결책:** 이러한 경우에는 실제 확률 분포 $p_{\text{data}}(x)$ 를 정확히 구하는 대신, '근사적으로(approximely)' 구하는 방법을 사용합니다 ($p_{\text{model}}(x) \approx p_{\text{data}}(x)$).

3. 생성 모델의 발전 과정

컴퓨터 비전 분야는 오랫동안 '인식'하고 '구별'하는 **분별 모델**을 중심으로 발전해 왔습니다. 하지만 새로운 이미지를 생성하는 기술도 꾸준히 연구되어 왔습니다.

- 초기 생성 모델:** Hidden Markov Model (HMM)이나 Boltzmann Machine 같은 초기 생성 모델들이 있었습니다. 그러나 이 모델들은 사람 얼굴이나 자연 영상처럼 복잡하고 사실적인 이미지를 생성하는 데는 한계가 있었습니다.
- GAN의 등장 (2014년):** 2014년에 ****생성 적대 신경망(Generative Adversarial Network; GAN)****이 발표되면서 생성 모델 분야에 혁명적인 발전이 일어났습니다. GAN은 자연스러운 이미지를 생성할 가능성을 크게 높였습니다.
- 텍스트-이미지 생성의 극대화 (예: DALL-E):** GAN의 등장을 시작으로 생성 모델은 엄청난 발전을 거듭했습니다. 이제는 "강아지 두 마리와 함께 강가를 달리는 소녀"와 같은 간단한 **텍스트를 입력하면, 그에 걸맞은 고품질의 이미지를 생성**하는 수준에까지 이르렀습니다.

4. 다양한 생성 모델 살펴보기

4.1. 가우시안 혼합 모델 (Gaussian Mixture Model; GMM)

- 원리:** 데이터의 '평균'과 '공분산(covariance) 행렬'을 이용해서 랜덤 데이터를 생성합니다. 예를 들어, 성인 남성의 키와 몸무게 데이터를 생성하는 데 활용될 수 있습니다.
- MNIST 숫자 영상 생성 사례:**
 - 28x28 픽셀의 MNIST 숫자 영상(손글씨 숫자 이미지)을 784차원의 벡터로 펼쳐서 가우시안 모델로 이미지를 생성할 수 있습니다.
 - 하지만 초기 가우시안 모델로 생성된 영상의 품질은 매우 낮았습니다.
- GMM의 개선:** 실제 데이터의 확률 분포는 단순한 가우시안 분포와는 많이 다릅니다. 이를 개선하기 위해 ****가우시안 혼합 모델(GMM)****은 여러 개의 가우시안 분포(k 개)를 섞어서 실제 분포에 더 가깝게 모델링합니다.
 - 최적의 매개변수 값은 주로 **기대-최대화(Expectation-Maximization; EM) 알고리즘**을 통해 찾아냅니다.
- GMM의 한계:** GMM을 사용해도 MNIST 숫자 영상을 생성할 수 있지만, 실제 세계의 복잡한 확률 분포는 가우시안 분포와는 여전히 차이가 크기 때문에, 생성된 이미지의 품질은 여전히 낮게 평가되었습니다.

4.2. 오토인코더(Autoencoder)를 이용한 생성 모델

- **원리:** 오토인코더는 입력 데이터를 받아서 그대로 출력으로 내보내도록 학습하는 신경망입니다. 마치 '압축했다가 다시 푸는' 과정과 같습니다.
 - **인코더(Encoder):** 입력 데이터를 압축하여 '잠복 공간(latent space)'이라는 곳으로 보냅니다.
 - **디코더(Decoder):** 잠복 공간의 압축된 정보를 다시 원본 데이터로 복원합니다.
- **생성 모델로서의 활용:** 오토인코더가 학습을 마치면, **인코더는 버리고 디코더만 사용합니다**. 잠복 공간에서 무작위로 벡터(숫자들의 조합)를 생성한 다음, 이 벡터를 디코더에 통과시키면 새로운 영상이 생성됩니다.

5. 생성 적대 신경망 (GAN): 이미지 생성의 혁신

GAN은 현재 이미지 생성 분야에서 가장 큰 영향력을 발휘하고 있는 모델 중 하나입니다.

5.1. GAN의 목표

- GAN의 목표는 실제 데이터의 확률 분포 $p_{data}(x)$ 에서 나온 데이터 샘플들을 가지고, 이 p_{data} 와 동일한(또는 매우 유사한) 새로운 샘플을 만들어내는 것입니다.

5.2. GAN의 동작 방식 (생성자와 판별자의 경쟁)

GAN은 마치 경찰과 위조지폐범처럼 서로 경쟁하며 발전하는 두 개의 신경망으로 구성됩니다.

1. 생성망 (Generator network; G):

- **역할:** 가짜 데이터를 생성하는 역할을 합니다.
- **작동 방식:** 간단한 분포(예: 가우시안 분포)에서 무작위로 '잠재적인 변수 z '를 뽑아냅니다. 이 z 를 생성망 G에 입력하면, G는 이를 실재처럼 보이는 가짜 데이터 $x = G(z)$ 로 변환하여 출력합니다. 이 가짜 데이터 x 는 생성망의 확률 분포 p_G 에서 나온 샘플이라고 할 수 있습니다. 생성망의 궁극적인 목표는 p_G 가 p_{data} 와 같아지도록 만드는 것입니다.

2. 분별망 (Discriminator network; D):

- **역할:** 입력된 데이터가 '진짜(real)'인지 아니면 생성망이 만든 '가짜(fake)'인지 구별하는 역할을 합니다.
- **작동 방식:** 실제 데이터와 생성망이 만든 가짜 데이터를 모두 입력받아, 각각을 '진짜' 또는 '가짜'로 분류하도록 훈련됩니다.

3. 적대적 훈련 (Adversarial Training):

- **경쟁과 발전:** 생성망 G와 분별망 D는 서로 '적대적'인 관계를 가지고 동시에 훈련됩니다.
 - 생성망 G는 분별망 D를 속여 자신의 가짜 데이터가 진짜라고 믿게 만들려고 노력합니다.
 - 분별망 D는 생성망 G가 만든 가짜 데이터를 정확히 구별하려고 노력합니다.
- **미니맥스(Minimax) 알고리즘:** 이러한 경쟁은 '미니맥스 알고리즘'이라는 수학적 원리에 따라 이루어지며, G와 D가 서로 발전하면서 최종적으로 생성망의 분포 p_G 가 실제 데이터의 분포 p_{data} 에 수렴하도록 만듭니다.

5.3. DC-GAN (Deep Convolutional GAN)

- **구조:** 2016년에 발표된 DC-GAN은 GAN의 생성망 G와 분별망 D를 모두 ****신경망(neural network)****으로 구성한 모델입니다.
- **성능 개선:** DC-GAN은 2014년에 발표된 초기 GAN보다 생성되는 영상의 품질이 크게 개선되었습니다.
- **최초의 자연 영상 생성:** DC-GAN은 **자연 영상(사람 얼굴, 풍경 등)**과 유사한 품질의 이미지를 최초로 성공적으로 생성한 모델로 평가됩니다.

5.4. GAN 방식의 장점과 단점

GAN은 지속적인 알고리즘 변형을 통해 생성 영상의 품질을 계속해서 개선해왔습니다.

• 장점:

- **간단한 구조:** 기본적인 GAN의 아이디어와 구조는 비교적 간단합니다.
- **매우 우수한 영상 품질:** 복잡하고 사실적인 이미지를 생성하는 데 탁월한 성능을 보여줍니다.

• 단점:

- **불안정한 훈련 과정:** 생성망과 분별망의 균형을 맞추기 어렵고, 훈련이 중간에 실패하거나 불안정해지는 경우가 많습니다.
- **대형 모델 확장 어려움:** 매우 큰 규모의 모델로 확장하고 학습시키는 데 어려움이 있습니다.